

Cooperative Service Caching for Reducing Delay in Multi-Edge Networks

Chang Kyung Kim , TaeYoung Kim , and SuKyoung Lee 

Abstract—Edge service caching enables computational tasks to be processed at nearby edge nodes (ENs), caching necessary programs rather than in the distant cloud. However, with the continually increasing popularity of IoT devices, resource-constrained ENs cannot ensure low-latency services for tasks generated by these devices. To address this issue, in this study, we propose a cooperative service caching mechanism that caches highly demanded services and processes tasks requiring those services across multiple ENs. Considering the waiting time for tasks to be processed at the EN, we formulate a delay minimization problem in cooperative service caching. To solve this problem, we devise an ant colony optimization-based algorithm. The simulation results show that the proposed method can achieve a lower total delay than existing methods.

Index Terms—Cooperative service caching, waiting time, total delay, multiple ENs, ant colony optimization.

I. INTRODUCTION

The Internet of Things (IoT) is making our lives smarter by integrating computing and communication technologies. Although IoT devices typically have limited computing resources, many IoT applications, such as autonomous vehicles and industrial robotics have strict delay requirements [1]. Therefore, edge computing (EC) has emerged as a computing and caching resource for applications in proximity [1]. Specifically, edge content caching stores multimedia videos, gaming data, and software update files on edge nodes (ENs) which are directly provided to users without computations [2]. In contrast, service caching stores essential programs or libraries needed to execute computational tasks for applications because the EN must utilize the relevant service to process the requested computational tasks [3], [4], [5], [6].

For instance, in a smart city, a vehicle requires computational resources for tasks such as traffic analysis, route optimization, and object detection, which are crucial for real-time applications such as augmented reality (AR) navigation and autonomous driving [2]. Because vehicles have limited computing capabilities, they may request the augmented classification and object recognition tasks for AR navigation to a nearby EN. To process requested tasks, the EN must cache the libraries for image processing and visual recognition models [7]. If the required service is not cached in the EN, the requested tasks are forwarded to the cloud, which degrades the vehicle's ability to navigate or drive autonomously due to extended communication delays.

Although some studies on edge service caching have focused on determining which services are cached on the EN, the limited caching resource of an individual EN, compared to the cloud, allows only a small number of services to be cached at a time [5].

Therefore, both industry and academia have explored cooperative multi-edge networks in which multiple ENs collaboratively provide

computing resources to multiple users [5], [8], [9], [10], [11], [12], [13]. Specifically, previous works have studied cooperative edge service caching strategies where multiple ENs collaboratively cache a set of services [5], [8], [10], [11], [13]. In these studies, if an EN is unable to process a task due to the absence of required services, it is expected to forward the received task to another EN with a cached service, whereas the waiting time of that EN is not considered. Consequently, if a highly demanded service is cached on an EN, all task requests requiring that service will be forwarded to that EN, leading to a scarcity of computing resources or extended waiting times, as tasks may need to wait for others to finish [9].

To address this issue, several studies have explored service replication, which caches the same services in multiple ENs to distribute task requests that require the same services [14], [15], [16], [17], [18]. In [14] and [17], a service replication strategy was proposed, enabling ENs to cache identical services, thereby allowing users to distribute tasks to multiple ENs that can be directly accessed from user devices. However, in cases where only a limited number of ENs are available in proximity to the user device, the tasks may not be processed within the ENs because of limited computing resources and cached services. To utilize more ENs, a few studies have suggested employing service replication schemes, in which requests are forwarded from the EN that directly receives the request from the user device to the processing EN via a predefined path [15], [16], [18]. However, an increase in requests that require the same services can lead to longer waiting times at ENs on a predefined path, resulting in an increase in the delay for the tasks to be processed.

In this paper, we propose a multi-edge cooperative service caching (MCSC) scheme, in which each EN forwards the task to neighboring ENs in a distributed manner until the task reaches the EN that can process it. Whenever an EN receives a request, it forwards the request when either of the two conditions is met: 1) the EN does not cache the required service for the task, or 2) the EN experiences a prolonged waiting time. Furthermore, to reduce waiting time, the MCSC also caches highly demanding services across multiple ENs to distribute task requests. We formulate a service caching problem that addresses how to cache services into multiple ENs to minimize the total delay caused by forwarding and processing requested tasks. For problem formulation, we investigate the waiting time for tasks to be processed and transmitted at the EN, which has not been considered in previous studies. To solve this problem, we develop an Ant Colony Optimization (ACO)-based service caching algorithm, as the problem formulation can be regarded as a facility location problem (FLP) [19]. The simulation results show that the proposed algorithm outperforms the benchmark schemes in terms of total delay.

II. SYSTEM MODEL AND PROBLEM FORMULATION

In 6G systems, wireless technologies such as multiple input multiple output (MIMO) and intelligent reflecting surface (IRS) have been the subject of recent research, with the objective of establishing wireless communication between ENs (such as Integrated Access and Backhaul) to improve cost efficiency and expand coverage beyond that of traditional wired networks [20]. Therefore, as illustrated in Fig. 1, we consider an edge network containing ENs $i \in I$, with each EN co-located with a small base station (SBS), where pairs of SBSs are connected via wireless links [19], [21]. Within this edge network, a MCSC controller was deployed to determine whether to cache and where to store each service [18], [19].

Received 30 August 2023; revised 21 April 2024; accepted 26 September 2024. Date of publication 7 October 2024; date of current version 14 February 2025. This work was supported by the National Research Foundation of Korea (NRF) Grant funded by Korean Government through the Ministry of Science and ICT (MSIT) under Grant 2022R1A2B5B01001683. The review of this article was coordinated by Dr. Qing Yang. (Corresponding author: SuKyoung Lee.)

The authors are with the Department of Computer Science, Yonsei University, Seoul 03722, Republic of Korea (e-mail: ckim48@yonsei.ac.kr; tykim92@yonsei.ac.kr; sklee@yonsei.ac.kr).

Digital Object Identifier 10.1109/TVT.2024.3475585

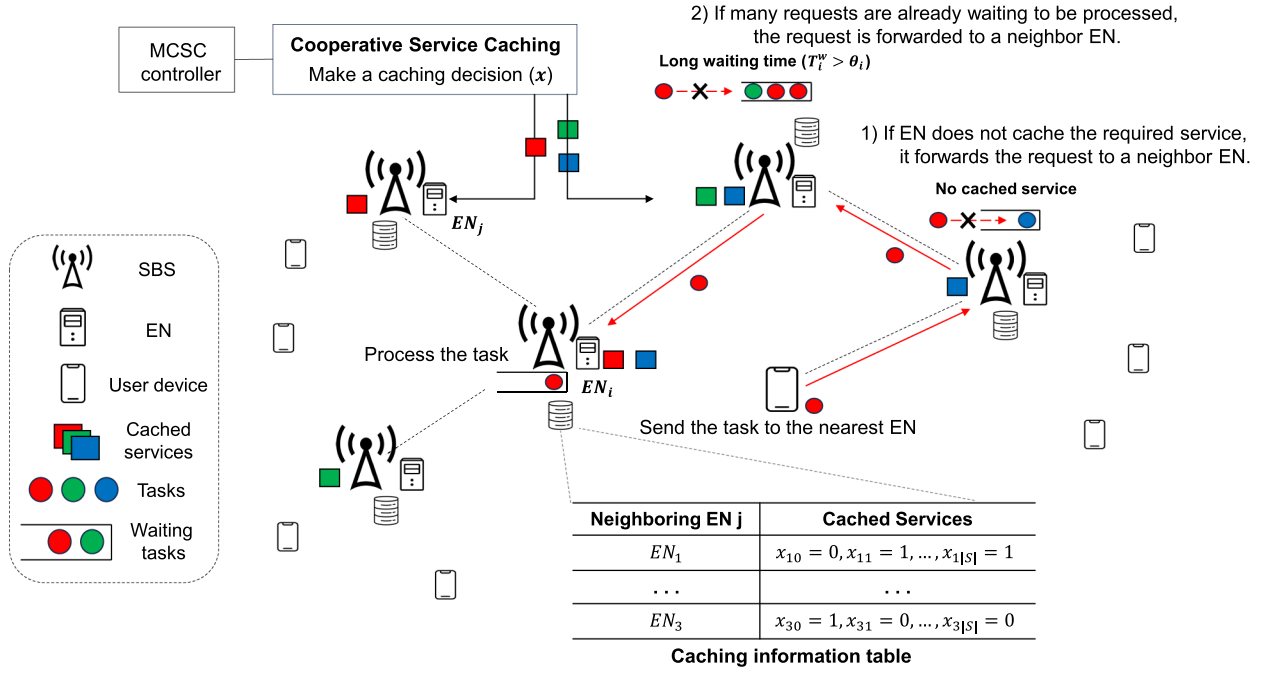


Fig. 1. Cooperative service caching and task forwarding in MCSC scheme.

Let S be the set of services required for the computation tasks requested by users. We introduce a binary variable x_{is} , defined such that $x_{is} = 1$ if EN i caches service $s \in S$, and 0 otherwise. After the caching decision is made, the controller caches services into the related ENs and sends the caching information to all ENs ($\in I$). This caching information, specifying which services are cached in neighboring ENs, is then stored as a table at each EN, as shown in Fig. 1. Then, when EN i receives a task request, if it does not cache the required service or its estimated processing time for the task, T_w^i exceeds a predefined threshold θ_i , and forwards the request to the neighboring EN, which caches the service by looking up the table. The processing time includes the waiting time at the EN and its average is estimated using an exponentially weighted moving average: $T_w^i = (1 - w) \cdot T_w^i(t - 1) + w \cdot T_w^i(t)$, where w is an exponential weight factor. If none of the ENs can process the task, the task is forwarded to the cloud, resulting in additional transmission time.

A. Transmission Time

We denote the transmission and interference sets of EN i as A_i and $\hat{A}_i$, respectively. A_i and $\hat{A}_i$ contain the ENs located within the ranges of d and $\hat{d}$ ($d < \hat{d}$). Then, link (i, j) exists only if EN j is in A_i , and the interference between ENs i and j occurs only if EN j is in $\hat{A}_i$.

We assume that each channel is an additive white Gaussian noise (AWGN) channel with constant noise power. Based on the deterministic power propagation model [19], the channel gain for link (i, j) is $|G_{ij}|^2 = \eta d_{ij}^{-\alpha}$, where α is the path loss exponent, η is an antenna related constant, and d_{ij} is the distance between ENs i and j . Then, the achievable data rate between ENs i and j can be written as

$$r_{ij} = W \log_2 \left(1 + \frac{P_{ij} \cdot |G_{ij}|^2}{\sigma^2 + \sum_{j' \in \hat{A}_i} P_{ij'} \cdot |G_{ij'}|^2} \right), \quad (1)$$

for $j \in A_i$, where σ is the Gaussian noise, W is the bandwidth of each channel, and P_{ij} is the transmission power from EN i to EN j .

After the proposed caching algorithm is performed, depending on the caching status and processing time of the request at EN i , the task

request can be forwarded up to L hops from EN i to another EN. Hence, the average arrival rate λ_i at EN i comprises of the incoming task requests from the users (λ_i^{req}) and other ENs (λ_i^{en}) in the edge system.

Let λ_{ij}^l be the arrival rate of task requests from EN j to EN i through l hops. Then, λ_i^{en} can be written as

$$\lambda_i^{en} = \sum_{l=1}^L \sum_{\forall j \in I, j \neq i} \lambda_{ij}^l. \quad (2)$$

Let K_i be the set of users connected directly to EN i . Given that each service is requested at rate q_s , we have

$$\lambda_i^{req} = \sum_{s \in S} |K_i| \cdot q_s \cdot x_{is}. \quad (3)$$

The probability that the request is processed at EN i is denoted as p_i , from (2) and (3), we can compute the average arrival rate of tasks that will be forwarded to another EN as:

$$\hat{\lambda}_i = (1 - p_i) \cdot (\lambda_i^{req} + \lambda_i^{en}). \quad (4)$$

When EN i sends the task to another EN, the average transmission rate r_i is

$$r_i = \frac{1}{|S|} \sum_{s \in S} \frac{\sum_{j \in A_i} r_{ij} x_{js}}{\sum_{j \in A_i} x_{js}}. \quad (5)$$

For wireless links, the arrival of task requests follows a Poisson distribution, and the service process follows a distribution [19]. Then, according to the Pollaczek-Khinchin formula of the M/G/1 queuing system, we obtain the average time during which each task request waits for EN i to be transmitted on the wireless link to be forwarded to EN j as follows:

$$T_i^r = \frac{F \hat{\lambda}_i + \rho_i \hat{\lambda}_i^2}{2(\rho_i - \hat{\lambda}_i)} + \frac{F}{r_i}, \quad (6)$$

where F is the average task size and ρ_i is the variance of task transmission time (i.e., r_i/F).

B. Total Delay

In the edge network, when a task request arrives at EN i requiring service s , the task can be computed at EN i , if the estimated processing time at EN i , T_i^w is less than a threshold θ_i . Otherwise, EN i forwards the task request to another EN caching the service s .

We model the processing of tasks at EN i as an M/M/1 queuing system with a processing time $1/\mu_i$. As $1/\mu_i$ follows an exponential distribution and T_i^w represents the sum of the processing times of all n task requests waiting at EN i , the probability density function for T_i^w follows a gamma distribution as $f_{T_i^w}(t) = \frac{\mu_i(\mu_i t)^{n-1} \cdot e^{-\mu_i t}}{(n-1)!} u(t)$ where $u(t)$ is an unit step function. Let p_n^i be the probability that n requests are waiting to be processed in EN i . Then, we can compute the probability p_i as follows:

$$\begin{aligned} p_i &= p[T_i^w < \theta_i] \\ &= \sum_{n=0}^{\infty} p[T_i^w < \theta_i] \cdot p_n^i \\ &= p_0^i + \sum_{n=1}^{\infty} \left[\int_0^{\theta_i} f_{T_i^w}(t) dt \right] \cdot p_n^i \\ &= \left(1 - \frac{\lambda_i}{\mu_i} \right) \cdot \left(1 + \sum_{n=1}^{\infty} \left[\int_0^{\theta_i} f_{T_i^w}(t) dt \right] \cdot \left(\frac{\lambda_i}{\mu_i} \right)^n \right), \end{aligned} \quad (7)$$

where λ_i is given as $\lambda_i = \lambda_i^{req} + \lambda_i^{en}$.

To compute λ_{ij}^l in (2), we calculate the rate of requests that are not processed by EN i but are forwarded to neighboring ENs within one hop:

$$\lambda_{i1} = \sum_{s \in S} |K_i| \cdot q_s \cdot (1 - x_{is}) + (1 - p_i) \cdot \lambda_i^{req}, \quad (8)$$

which follows $\lambda_{i2} = (1 - p_i) \cdot \lambda_{i1}$ to $\lambda_{iL} = (1 - p_i) \cdot \lambda_{i(L-1)}$. From (8), we obtain λ_{ij}^l as follows:

$$\lambda_{ij}^l = \sum_{s \in S} \lambda_{jl} \cdot \frac{1}{\sum_{j' \in A_j} x_{j's}}, \quad 0 \leq l \leq L. \quad (9)$$

We denote the processing time for the service s at EN i as T_{is}^{proc} which is given by $T_{is}^{proc} = \frac{m_s \cdot F}{f_i}$, where m_s is the computational resource required for processing the service s in CPU cycle and f_i is the computing capability of i . Let T_c be the time required to send the task to the cloud. Then, we can recursively express the expected time for processing the task sent via l hops to EN i as:

$$\begin{aligned} T_{is}(l) &= (1 - p_i)[(1 - v_l) \cdot T_c + v_l \cdot \{T_i^r + T_{is}(l+1)\}] \\ &\quad + p_i \cdot \left(T_{is}^{proc} + \frac{p_i \cdot \frac{\lambda_i}{\mu_i}}{\mu_i - p_i \cdot \lambda_i} \right), \quad 0 \leq l \leq L, \end{aligned} \quad (10)$$

where $v_l = 1$ if $l < L$, otherwise 0. Then, the total delay can be expressed as:

$$T_{is} = \frac{F}{r} + T_{is}(0), \quad (11)$$

where r is the average transmission rate from the user to the nearest EN.

We formulate the caching problem to minimize the total delay as follows:

$$\min_{\mathbf{x}} \sum_{i \in I} \sum_{s \in S} T_{is}, \quad (12)$$

$$\text{s.t. } 0 < \sum_{s \in S} b_s \cdot x_{is} \leq C_i, \quad (13)$$

Algorithm 1: MCSC Algorithm.

Input: $\lambda_i^{req}, \mu_i, F, r_i$, **Output:** Caching Decision $\mathbf{x}$

- 1: $\tau_{is}^a(0) = \frac{1}{T_{is}}$ ▷ Initialize pheromone value
- 2: **for** $t = 1$ to t_{max} **do**
- 3: **for** $a = 1 : |A|$ **do**
- 4: Generate a caching decision $\mathbf{x}_a$ using the probability $p_{is}^a(t)$ in Eq. (14)
- 5: $I' = \{i | i \in I, \sum_{s \in S} b_s \cdot x_{is}^a > C_i\}$
- 6: $I' = \text{Sort } I' \text{ in descending order}$
- 7: **for** $i \in I'$ **do**
- 8: $s = \underset{s \in S, x_{is}^a = 1}{\operatorname{argmin}} e_s; x_{is}^a = 0$
- 9: $I_c = \{i | i \in I, x_{is}^a = 0, \sum_s b_s x_{is}^a + b_s \leq C_i\}$
- 10: **if** $I_c \neq \emptyset$ **then** $i' = \underset{i' \in I_c}{\operatorname{argmin}} \{\lambda_{i'}^{req}\}; x_{i's}^a = 1$
- 11: **end for**
- 12: Evaluate $\mathbf{x}_a$
- 13: **if** $f(\mathbf{x}_a) > f(\mathbf{x})$ **then** $\mathbf{x} = \mathbf{x}_a$
- 14: Update τ_{is}^a
- 15: **end for**
- 16: **end for**

where b_s denotes the caching capacity required by the service s . Equation (13) is a constraint that indicates that the total size of services cached at EN i cannot exceed the maximum caching capacity of EN i , C_i . The formulated problem can be proven to be NP-hard through reduction in FLP which has already been proven to be NP-hard. This reduction can be achieved by viewing ENs, services, and requests from users as candidate locations, facilities, and market demands, respectively.

III. PROPOSED MCSC ALGORITHM

ACO is a meta-heuristic algorithm that has proven to be effective in solving NP-hard problems such as FLP and Knapsack problems [19]. Determining the best locations for facilities based on the geographical cost of maximizing user profits is similar to caching services into ENs based on service demand to minimize the total delay. Therefore, we designed an ACO-based MCSC algorithm to minimize the total delay.

In our algorithm based on ACO, a set of ants A collaborate to solve the caching problem in (12) and (13). At each iteration, each ant $a \in A$ releases a pheromone along its path that is proportional to the quality of the solution. Subsequently, the other ants will have a higher probability of following the path with a higher amount of pheromones. The pheromone is updated every iteration t based on the quality of the solution. The proposed algorithm terminates after t_{max} iterations. The overall process of the MCSC algorithm is presented in Algorithm 1.

As shown in Algorithm 1, each ant a first constructs the caching decision $\mathbf{x}_a = [x_{00}^a, x_{01}^a, \dots, x_{10}^a, x_{11}^a, \dots, x_{|I||S|}^a]$, where $x_{is}^a = 1$ if ant a decides to cache service s in to EN i , $x_{is}^a = 0$ otherwise. To construct the caching decision, each ant a decides to cache service s at EN i in the t -th iteration with probability, $p_{is}^a(t)$ (line 5), which is given by

$$p_{is}^a(t) = \frac{(\tau_{is}^a(t))^\alpha (h_{is})^\beta}{\sum_{i \in I} (\tau_{is}^a(t))^\alpha (h_{is})^\beta}, \quad (14)$$

where $\tau_{is}^a(t)$ is the pheromone value of ant a , h_{is} is the heuristic information, and α and β are the parameters for controlling the impact of the pheromone and heuristic information on the caching decision.

The initial caching decision, probabilistically generated by ant a as a local solution $\mathbf{x}_a(t)$, may require regeneration to meet the constraint of (13), leading to extended convergence times to achieve a global

solution. Hence, the proposed MCSC algorithm locally optimizes this initial caching decision $\mathbf{x}_a$, aiding the ant's rapid convergence towards the global solution. For this, the proposed algorithm finds the ENs that does not meet the caching storage constraint and sort them by the request arrival rates from IoT users λ_i^{req} (33) in descending order, denoted as I' (lines 5-6). Now, we introduce caching efficiency e_s , which is defined as the ratio of the request rate for service s to its size, and can be calculated as follows:

$$e_s = \frac{|K_i| \cdot q_s}{b_s}. \quad (15)$$

For each EN $i \in I'$, the algorithm finds the service s with the lowest caching efficiency e_s (line 8). Next, among the set of ENs $i' \in I_c$ that do not cache the found service s and have sufficient space to cache it, the algorithm caches the service s into EN i' with the lowest arrival rate from IoT users, $\lambda_{i'}^{req}$ (lines 10-14).

At the end of each iteration t , $f(\mathbf{x}_a(t))$ is calculated using Algorithm 2. Then, the ant a updates its caching decision $\mathbf{x}$ only if $f(\mathbf{x}_a(t))$ is higher than $f(\mathbf{x})$ (lines 8-11) because $f(\mathbf{x})$ is designed as the reciprocal of the total delay $\sum_{i \in I} \sum_{s \in S} T_{is}$ in (12).

Each ant a then evaluates the performance of $\mathbf{x}_a(t)$ using the performance evaluation function, $f(\mathbf{x}_a(t))$, which is given by

$$f(\mathbf{x}_a(t)) = \frac{1}{\sum_{i \in I} \sum_{s \in S} T_{is}(\mathbf{x}_a(t))}. \quad (16)$$

Next, ant a updates its pheromone value for the next iteration $t + 1$ (line 11) as follows:

$$\tau_{is}^a(t+1) = (1 - \rho) \tau_{is}^a(t) \prod_{a \in A} \gamma_a(t), \quad (17)$$

where ρ is the evaporation rate of the pheromone ($\rho \in [0, 1]$) and

$$\gamma_a(t) = \frac{f(\mathbf{x}_a(t))}{\sum_{a \in A} f(\mathbf{x}_a(t))}, \quad (18)$$

which signifies how good the performance of $\mathbf{x}_a(t)$ comparing to the caching decisions made by other ants.

Noting that heuristic information is used along with pheromone to make probabilistic caching decisions, we design it as a function of the total delay at EN i , $T_i = \sum_{s \in S} T_{is}$ and the demand for service s , ω_s . Let λ_{is} denote the total arrival rate of tasks requiring service s at EN i . Then, from (8), we can obtain the demand for service s as

$$\omega_s = \frac{\sum_{i \in I} \lambda_{is}}{\sum_{i \in I} \sum_{s \in S} \lambda_{is}}, \quad (19)$$

where λ_{is} as $\lambda_{is} = \sum_{l=1}^L \sum_{j \in I, j \neq i} \lambda_{jl} \cdot \frac{1}{\sum_{j' \in A_j} x_{j's}}$. Using (19), we express heuristic information h_{is} as follows:

$$h_{is} = \frac{\omega_s}{T_i(\mathbf{x}_a)}. \quad (20)$$

The proposed ACO algorithm for solving the proposed problem is described in Algorithm 1, and the algorithm for calculating $f(\mathbf{x}_a(t))$ in (16) is shown in Algorithm 2.

The time complexity of the proposed MCSC is $O(t_{\max} * |A|)$ whereas additional $|I|$ iterations are required to adjust the caching decision when it does not satisfy the caching storage constraint of EN.

IV. PERFORMANCE EVALUATION

In this section, we evaluate the convergence performance of the proposed MCSC scheme. Then, in terms of the total delay, the performance of MCSC is evaluated, and compared with two benchmark methods: 1)

Algorithm 2: Performance Evaluation of Caching Decision.

Input: $\mathbf{x}_a$, **Output:** Caching Decision $f(\mathbf{x}_a(t))$.

$\mathbf{p}(k) \leftarrow$ random floating number in $[0, 1]$, where

$\mathbf{p} = [p_0, p_1, \dots, p_{|I|}]$

2: $k \leftarrow 0$ ▷ Initialize a number of iterations

repeat

4: Calculate $\lambda_{en}^*(k)$ with $\mathbf{p}(k)$ using Eq. (2)

Calculate $\mathbf{p}^*(k)$ with obtained $\lambda_{en}^*(k)$ using Eq. (7)

6: $\mathbf{p}(k+1) \leftarrow \mathbf{p}^*(k)$

$k \leftarrow k+1$

8: **until** convergence or a maximum number of iterations is reached

TABLE I
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
P_{ij}	400 mW	σ, η, e	0.1, 1, 3
$d, \hat{d}$	200 m, 150 m	$ S $	20
F	10 MB	θ_i	200 ms
C_i, b_s	160 GB, 80 GB	f_i, m_s	4 GHZ, 50 cycles/bit

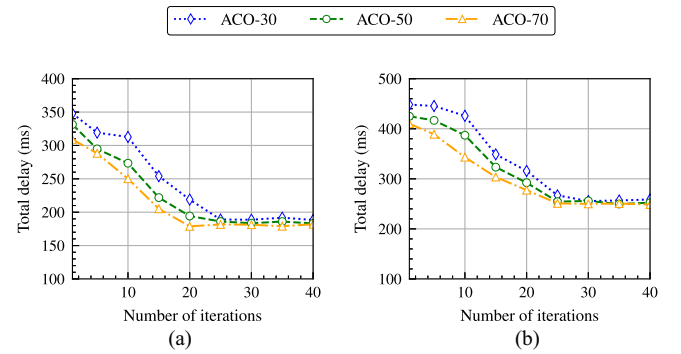


Fig. 2. Convergence performance when there are 300 and 500 users. (a) 300 users. (b) 500 users.

Non-Cooperative (n-CSC): each EN caches the most requested service independently [7], [22]; 2) CSC: a caching decision is made such that the total delay is minimized. When making a decision, if the EN considered cannot cache the service, one of its neighbors is selected [5], [10], [11]. In CSC, the task can be processed either at the EN directly connected to the device or at the neighboring ENs of that EN. For all three algorithms, the request is sent to the remote cloud if the necessary services for the requests are not cached in any EN. The transmission rate from the EN to the cloud is 10 MB/s [23], assuming that the storage capacity and computing capability of the cloud are sufficient. For the simulations, we consider an area of 1 km $\times$ 1 km, where 20 ENs are uniformly deployed and 100 to 500 users with a step of 50 are randomly distributed. The request rate for service q_s follows a Zipf distribution with a parameter of 0.7 [7]. The remaining parameters used in the simulations are listed in Table I.

Fig. 2 illustrates the convergence performance of the MCSC algorithm. As shown in the figure, the proposed algorithm converges to a quasi-optimal solution in up to a maximum of 30 iterations for both 300 and 500 users, even if the convergence speed decreases as the number of ants increases from 30 to 50 to 70.

Fig. 3 shows the total delay versus the number of users for bandwidths of 10 and 15 MHz for different values of L , 2, 4, and 6 in MCSC.

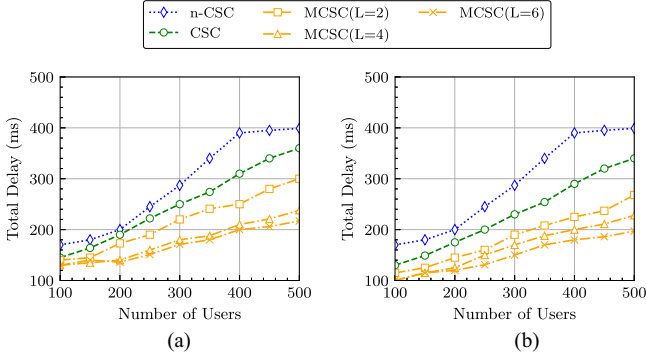


Fig. 3. Total delay with respect to the number of users when $W = 10$ and 15 MHz. (a) $W = 10$ MHz. (b) $W = 15$ MHz.

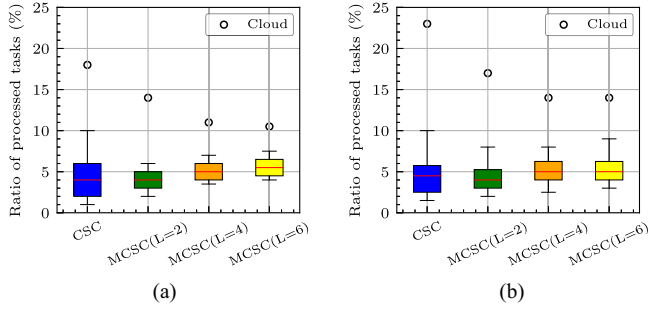


Fig. 4. The ratio of tasks processed by the ENs and cloud out of the total number of tasks requested. (a) 300 users. (b) 500 users.

Accordingly, the total delay increases with the increase in the number of users for the three algorithms, resulting in more task requests to the edge. Both CSC and MCSC achieve lower delays for 15 MHz than for 10 MHz, whereas the delay of the n-CSC is not affected by the bandwidth because the n-CSC is not supposed to forward the tasks to other ENs. We also observe from Fig. 3 that the total delay decreases with a larger L . This is because with a larger L , if the ENs cannot process the tasks, they will have a higher chance of forwarding the requests to other ENs rather than relying on the cloud. This is validated in Fig. 4, which plots the distribution of the ratio of tasks processed by the ENs and the cloud to the total number of tasks requested by the users for the three algorithms when $W = 10$ MHz. For the same reason, in the n-CSC and CSC algorithms, more tasks are sent to the distant cloud due to the absence of service, compared to MCSC, as can be observed from Fig. 4. We also see in Fig. 4 that the three algorithms send more tasks to the cloud for 500 users than for 300 users, as more tasks are requested with an increase in the number of users, whereas for both numbers of users, MCSC processes more tasks at the EN compared to the other two algorithms.

Fig. 5 plots the number of service types versus the number of ENs caching the same service. In the n-CSC, 17 types of services are not cached in any EN, whereas five and six types of services are cached in CSC and MCSC, respectively. Hence, more requests are sent to the cloud due to the absence of services at ENs in n-CSC, as shown in Fig. 4, resulting in a longer total delay compared with CSC and MCSC. Although CSC caches one more type of service than MCSC, MCSC caches frequently requested services in more ENs than CSC. For example, MCSC caches the two most requested services in five ENs, whereas CSC caches these services in three ENs as illustrated in Fig. 5.

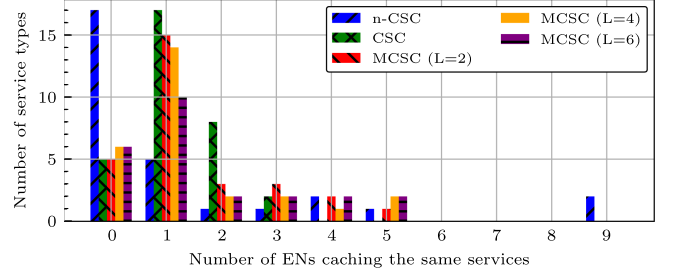


Fig. 5. The number of service types versus the number of ENs caching the same service.

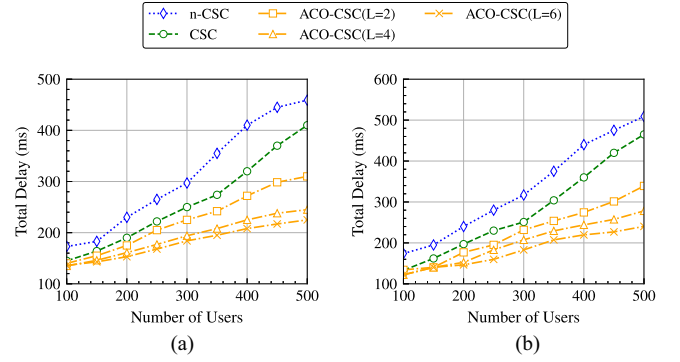


Fig. 6. Total delay with respect to number of users when Zipf parameter for q_s is 0.8 and 1.0. (a) $Zipf = 0.8$. (b) $Zipf = 1.0$.

Therefore, in CSC, a greater number of task requests necessitating these cached services are sent to the cloud due to fewer ENs that can process for services leading to a longer total delay than MCSCs.

Fig. 6 shows the total delay for q_s with two different Zipf parameters: 0.8 and 1 when $W = 10$ MHz [7], [15]. As shown in Fig. 6, the delay performance of the proposed algorithm improves relative to the other benchmark methods as the Zipf parameter increases. Specifically, when the Zipf parameter is raised from 0.8 to 1.0, the delay for MCSC, in comparison with CSC, improves by an average of 20.4%. This is because, unlike CSC, the proposed algorithm enables the processing of computational tasks requiring the same highly demanded services on an additional one or two ENs on average, thereby reducing the waiting times at each EN.

V. CONCLUSION

In this paper, we propose the multi-edge cooperative service caching (MCSC) scheme that caches frequently requested services in multiple ENs, enabling tasks requiring the same services to be processed across multiple ENs. Because the waiting time at ENs can influence the total delay in processing task requests in service caching, we formulated a service caching problem with the objective of minimizing total delay by considering the waiting time for tasks to be processed and transmitted at the EN. As the proposed problem is NP-hard, we proposed an ACO-based service caching algorithm to obtain a solution. The simulation results showed that the proposed method achieves a better total delay than the other two benchmark methods by reducing the waiting time at each EN as well as the number of tasks sent to the cloud.

In future studies, considering the mobility of IoT users, particularly vehicles, it would be interesting to explore how to cache the required services and process related tasks along vehicles' paths to reduce the

total delay. Furthermore, designing a prediction model for vehicle mobility, with the aim of determining the optimal EN for service caching, is also worth considering.

REFERENCES

- [1] W. Rafique, A. S. Hafid, and S. Cherkaoui, "Complementing IoT services using software-defined information centric networks: A comprehensive survey," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23545–23569, Dec. 2022.
- [2] Z. Xue, C. Liu, C. Liao, G. Han, and Z. Sheng, "Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6709–6722, May 2023.
- [3] X. Dai et al., "Task offloading for cloud-assisted fog computing with dynamic service caching in enterprise management systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 662–672, Jan. 2023.
- [4] J. Bi, H. Yuan, K. Zhang, and M. Zhou, "Energy-minimized partial computation offloading for delay-sensitive applications in heterogeneous edge networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 10, no. 4, pp. 1941–1954, Oct.–Dec. 2022.
- [5] T. Zhou, Y. Yue, D. Qin, X. Nie, X. Li, and C. Li, "Mobile device association and resource allocation in HCNs with mobile edge computing and caching," *IEEE Syst. J.*, vol. 17, no. 1, pp. 976–987, Mar. 2023.
- [6] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.
- [7] S.-W. Ko, S. J. Kim, H. Jung, and S. W. Choi, "Computation offloading and service caching for mobile edge computing under personalized service preference," *IEEE Trans. Wirel. Commun.*, vol. 21, no. 8, pp. 6568–6583, Aug. 2022.
- [8] Z. Xu et al., "Near-optimal and collaborative service caching in mobile edge clouds," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 4070–4085, Jul. 2023.
- [9] C. Barrios and M. Kumar, "Service caching and computation reuse strategies at the edge: A survey," *ACM Comput. Surv.*, vol. 56, no. 2, 2023, Art. no. 43, doi: [10.1145/3609504](https://doi.org/10.1145/3609504).
- [10] R. Chen, H. Lu, and P. Ma, "User-centric cooperative MEC service offloading," in *Proc. 2021 IEEE Wireless Commun. Netw. Conf.*, 2021, pp. 1–6.
- [11] S. Bi, L. Huang, and Y.-J. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wirel. Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.
- [12] ETSI, "Multi-access edge computing (MEC); Requirements and reference architecture," ETSI GS MEC 002 V3.2.1, Feb. 2024. [Online]. Available: https://www.etsi.org/deliver/etsi_gs/MEC/001_099/002/03.02.01_60/gs_MEC002v030201p.pdf
- [13] X. Kong, Y. Wu, H. Wang, and F. Xia, "Edge computing for Internet of Everything: A survey," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23472–23485, Dec. 2022.
- [14] K. Li, M. Tao, and Z. Chen, "Exploiting computation replication for mobile edge computing: A fundamental computation-communication tradeoff study," *IEEE Trans. Wirel. Commun.*, vol. 19, no. 7, pp. 4563–4578, Jul. 2020.
- [15] D. Ren, X. Gui, and K. Zhang, "Adaptive request scheduling and service caching for MEC-Assisted IoT networks: An online learning approach," *IEEE Internet Things J.*, vol. 9, no. 18, pp. 17372–17386, Sep. 2022.
- [16] R. Li, Z. Zhou, X. Zhang, and X. Chen, "Joint application placement and request routing optimization for dynamic edge computing service management," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 12, pp. 4581–4596, Dec. 2022.
- [17] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec., 2023.
- [18] M. K. Somesula, S. K. Mothku, and S. C. Annadanam, "Cooperative service placement and request routing in mobile edge networks for latency-sensitive applications," *IEEE Syst. J.*, vol. 17, no. 3, pp. 4050–4061, Sep. 2023.
- [19] P. Han, Y. Liu, and L. Guo, "Interference-aware online multicomponent service placement in edge cloud networks and its AI application," *IEEE Internet Things J.*, vol. 8, no. 13, pp. 10557–10572, Jul. 2021.
- [20] Y. Sun, B. Xie, S. Zhou, and Z. Niu, "MEET: Mobility-enhanced edge intelligence for smart and green 6G networks," *IEEE Commun. Mag.*, vol. 61, no. 1, pp. 64–70, Jan. 2023.
- [21] H. Zhang, Y. Yang, B. Shang, and P. Zhang, "Joint resource allocation and multi-part collaborative task offloading in MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8877–8890, Aug. 2022.
- [22] G. Zhang, S. Zhang, W. Zhang, Z. Shen, and L. Wang, "Joint service caching, computation offloading and resource allocation in mobile edge computing systems," *IEEE Trans. Wirel. Commun.*, vol. 20, no. 8, pp. 5288–5300, Aug. 2021.
- [23] Z. Hong, W. Chen, H. Huang, S. Guo, and Z. Zheng, "Multi-hop cooperative computation offloading for industrial IoT–edge–cloud computing environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 12, pp. 2759–2774, Dec. 2019.